

OutOfTheDark – Code-Dokumentation

Dieses Dokument beschreibt jede C#-Datei im Projekt OutOfTheDark (Unity-2D-Lichtpuzzle-Spiel): was die Datei macht und welche Klassen, Felder und Methoden am wichtigsten sind. Die Skripte sind nach ihrem Ordner unter Assets/Scripts/ gruppiert.

Ordner: Core

Core/AudioManager.cs

Persistenter Audio-Singleton (DontDestroyOnLoad), der Musik-, Ambient- und SFX-Lautstärke über PlayerPrefs speichert und für jedes Spielereignis einen benannten One-Shot-Sound-Hook bereitstellt. Die AudioClip-Felder sind bewusst leer (keine Audiodateien verfügbar), jeder Play-Aufruf ist null-sicher, sodass das Spiel auch ohne echte Clips lautlos, aber voll funktionsfähig bleibt.

Wichtige Code-Elemente:

- Klasse: AudioManager : MonoBehaviour
- Felder: Instance (static Singleton); menuMusic, gameplayMusic, ambientLoop (AudioClip); shotFired, reflection, teleport, switchClick, goalReached, levelComplete, levelFailed, menuClick (AudioClip); MusicVolume, SfxVolume (float, aus PlayerPrefs)
- Methoden: SetMusicVolume/SetSfxVolume (setzt & persistiert Lautstärke); PlayMusic/PlayAmbient (startet Loop-Sources); PlaySfx (Einzel-Sound); PlayShotFired, PlayReflection, PlayTeleport, PlaySwitchClick, PlayGoalReached, PlayLevelComplete, PlayLevelFailed, PlayMenuClick (benannte Kurzform-Wrapper)

Core/BrightnessOverlay.cs

Persistentes, halbtransparentes schwarzes UI-Overlay, das die Bildschirmhelligkeit basierend auf einer gespeicherten Einstellung abdunkelt. Bewusst getrennt von ScreenFader, damit sich beide Komponenten nicht um dieselbe Alpha-Kontrolle streiten.

Wichtige Code-Elemente:

- Klasse: BrightnessOverlay : MonoBehaviour
- Felder: Instance (static Singleton)
- Methoden: Apply(float brightness) – berechnet aus dem Helligkeitswert (0–1) einen Verdunkelungs-Alpha (max. 0.85) und setzt die Overlay-Farbe

Core/InteractionResult.cs

Definiert die Datenstrukturen, mit denen LightInteractable-Objekte (Spiegel, Prismen, Filter usw.) mitteilen, was mit einem auftreffenden Lichtstrahl geschehen soll.

Wichtige Code-Elemente:

- Enum: RayAction – Redirect, Split, Absorb, PassThrough, Teleport
- Struct: ChildRaySpec – Blaupause für einen Kindstrahl (Direction, Color, Intensity)
- Struct: InteractionResult – Ergebnis von ProcessRay (Action, NewDirection, NewColor?, NewIntensity?, ChildRays, TargetPosition) mit statischen Factory-Methoden Redirect, Absorb, PassThrough, Split, Teleport

Core/KeyBindings.cs

Kleine, statische Registry für neu belegbare Tasten, aktuell nur für die Pause-Aktion, gestützt auf PlayerPrefs.

Wichtige Code-Elemente:

- Klasse: KeyBindings (static)
- Property: PauseKey (KeyCode, liest/schreibt PlayerPrefs, Default Escape)

Core/LevelIds.cs

Zentrale Sammlung von Szenennamen-Konstanten statt Magic Strings.

Wichtige Code-Elemente:

- Klasse: LevelIds (static)
- Felder: MainMenu, LevelSelect, Settings; Level01–Level06; AllLevels (readonly string[])
- Methode: GetNextLevel(currentSceneName) – liefert den Namen des nächsten Levels oder null

Core/LightInteractable.cs

Abstrakte Basisklasse für alle Objekte, auf die ein LightRay reagieren kann (Spiegel, Linsen, Prismen, Filter, Absorber usw.). Bündelt die Debounce-Logik (verhindert Mehrfachauslösung) und "Nudge"-Logik (kleine Positionsverschiebung nach Reflexion) und delegiert die fachliche Entscheidung an die abstrakte Methode ProcessRay.

Wichtige Code-Elemente:

- Klasse: LightInteractable : MonoBehaviour (abstract, benötigt Collider2D)
- Felder: ignoreDuration, nudgeDistance (float, serialized)
- Methoden: OnTriggerEnter2D (erkennt Treffer, ruft ProcessRay auf, wendet Ergebnis an); ApplyResult (setzt Redirect/Split/Absorb/PassThrough/Teleport um, inkl. Audio/VFX); ProcessRay (abstract, von Subklassen implementiert); RotateVector (static Hilfsfunktion); BuildFan (erzeugt fächerförmige Kindstrahl-Specs, genutzt von Splitter und Crystal)

Core/LightRay.cs

Der physikalisch simulierte Lichtstrahl-Projektil-Typ: ein kinematischer Rigidbody2D, der sich geradlinig bewegt, bis ein LightInteractable ihn umlenkt, aufspaltet, absorbiert oder teleportiert. Kümmt sich um Bewegung, Darstellung (Glow-Sprite, Pulsieren, Trail), Distanzabschwächung und Lebensdauer-Failsafes.

Wichtige Code-Elemente:

- Klasse: LightRay : MonoBehaviour
- Felder: speed, direction, color, intensity (0–1), maxLifetime, maxRange, attenuationConstant/Linear/Quadratic, minVisibleIntensity, generation, sourcePrefab, glowRenderer, glowScale, pulsateSpeed, pulsateAmount
- Methoden: Awake (richtet Rigidbody2D, Collider2D, SpriteRenderer, Light2D, TrailRenderer ein); FixedUpdate (Bewegung, Attenuation, Terminierung); OnTriggerEnter2D/OnCollisionEnter2D (Terminierung bei "AbsorbWand"); SetDirection/GetDirection; SetSpeed; SetColor/GetColor; SetIntensity/GetIntensity; ApplyVisuals (Pulsieren + Farbe/Alpha auf Sprite/Glow/Light2D/Trail); Terminate; SpawnChild (Kindstrahl, max. Generation 4); CreateAndInitialize (static Factory für den initialen, vom Spieler abgefeuerten Strahl)

Core/OpticalMaterial.cs

Datengetriebenes ScriptableObject für optische Materialeigenschaften (Glas, Wasser, Diamant usw.).

Wichtige Code-Elemente:

- Klasse: OpticalMaterial : ScriptableObject
- Felder: refractiveIndex (Brechungsindex n); absorptionCoefficient (Beer-Lambert); shininess (Blinn-Phong-Glanz); dispersionOffsetRGB (Pro-Kanal-Offset für Dispersion)

Core/OpticsMath.cs

Sammlung gemeinsam genutzter Optik-/Beleuchtungsformeln.

Wichtige Code-Elemente:

- Klasse: OpticsMath (static)
- Methoden: Reflect (Reflexionsvektor); Refract (Snellius'sches Brechungsgesetz, liefert false bei Totalreflexion); FresnelSchlick (Fresnel-Näherung); LambertDiffuse; BlinnPhongSpecular; BeerLambertTransmittance; PointLightAttenuation

Core/ProgressManager.cs

Statischer, PlayerPrefs-gestützter Fortschritts-Tracker: Level-Abschluss, Sterne, Bestzeit, geringster Munitionsverbrauch. Ein Level ist freigeschaltet, sobald das vorherige abgeschlossen ist.

Wichtige Code-Elemente:

- Klasse: ProgressManager (static)
- Methoden: IsUnlocked, IsCompleted, GetStars, GetBestTime, GetBestUsedBalls, RecordCompletion (speichert nur, wenn besser als Bestwert), CalculateStars (1–3 je nach Munitionsverbrauch), ResetAllProgress

Core/RadialGradientTexture.cs

Statische Utility, die einmalig (gecacht) ein weiches Radial-Gradient-Sprite für das Glow-Halo sowie ein Trail-Material erzeugt. Alpha-Verlauf von opak zu transparent wirkt auf schwarzem Hintergrund wie additives Blending, ohne eigenen Shader.

Wichtige Code-Elemente:

- Klasse: RadialGradientTexture (static)
- Methoden: GetGlowSprite (64×64 radialer Alpha-Falloff); GetTrailGradientTexture (1×64 vertikaler Alpha-Verlauf für Trail-Querschnitt); GetTrailMaterial (klont Template-Material, setzt Trail-Gradient als Textur)

Core/ScreenFader.cs

Persistentes Vollbild-Fade-Overlay, blendet nach jedem Szenenladevorgang von Schwarz ein. Nutzt unskalierte Zeit, damit es auch bei `Time.timeScale = 0` funktioniert.

Wichtige Code-Elemente:

- Klasse: ScreenFader : MonoBehaviour
- Felder: Instance (static), fadeDuration
- Methoden: OnEnable/OnDisable (Scene-Loaded-Event); OnSceneLoaded; Fade(from, to) (Coroutine, animiert Overlay-Alpha)

Core/VfxManager.cs

Persistenter VFX-Singleton, feuert kurzlebige Partikel-Bursts für Strahl-Ereignisse ab (Einschlag, Absorption, Teleport, Ziel erreicht). Effekt-Prefabs sind optional und null-sicher.

Wichtige Code-Elemente:

- Klasse: VfxManager : MonoBehaviour
- Felder: Instance (static); sparkEffectPrefab, absorbEffectPrefab, teleportEffectPrefab, goalEffectPrefab (ParticleSystem)
- Methoden: SpawnSpark, SpawnAbsorb, SpawnTeleport, SpawnGoalBurst, Spawn (privat, instanziiert & zerstört automatisch)

EnvironmentController.cs

(Liegt direkt in Assets/Scripts/.) Konfiguriert zur Laufzeit und im Editor ([ExecuteAlways]) die grundlegende Szenenbeleuchtung: Kamera-Hintergrundfarbe, flaches Umgebungslicht, kann optional alle Szenenlichter deaktivieren, um die dunkle, nur durch Lichtstrahlen erhellte Optik sicherzustellen.

Wichtige Code-Elemente:

- Klasse: `EnvironmentController` : `MonoBehaviour` (`ExecuteAlways`)
- Felder: `backgroundColor`, `ambientColor`, `ambientIntensity`, `disableSceneLights`
- Methoden: `Start/OnValidate` (rufen `ApplyEnvironmentSettings` auf); `ApplyEnvironmentSettings` (Kamera + `RenderSettings`); `DisableAllSceneLights`

Ordner: Interactables

Alle Dateien in diesem Ordner sind Objekte, mit denen ein `LightRay` interagieren kann, und erben (fast alle) von der Basisklasse `LightInteractable` (siehe oben), deren abstrakte Methode `ProcessRay(LightRay ray, Vector2 hitPoint, Vector2 hitNormal)` sie überschreiben, um ihr eigenes optisches/Gameplay-Verhalten festzulegen.

Interactables/AreaRevealer.cs

Deckt beim ersten Treffer dauerhaft einen kreisförmigen Bereich des Fog-of-War auf und lässt den Strahl unverändert weiterlaufen. Aktiviert zusätzlich eine eigene `Light2D`-Komponente. Gemeinsame Basis für `Light Crystals`, `Glow Stones` und `Light Towers`.

Wichtige Code-Elemente:

- Klasse: `AreaRevealer` : `LightInteractable`
- Felder: `revealRadius`, `fadeDuration`, `litColor`, `ownLight` (`Light2D`)
- `ProcessRay`: deckt Fog auf, färbt Sprite um, schaltet `ownLight` ein (nur beim ersten Treffer), gibt immer `PassThrough()` zurück

Interactables/BlackHole.cs

Gravitationsartiges Feld, das nahegelegene Strahlen kontinuierlich zum Zentrum hin verbiegt; Strahlen, die zu nah kommen, werden zerstört. Erbt NICHT von `LightInteractable` (kein `ProcessRay`), da es Strahlen über mehrere Frames beeinflusst.

Wichtige Code-Elemente:

- Klasse: `BlackHole` : `MonoBehaviour`
- Felder: `pullRadius`, `pullStrength`, `eventHorizonRadius`
- `FixedUpdate`: sucht Strahlen im `pullRadius` per `Physics2D.OverlapCircleAll`, zerstört sie innerhalb `eventHorizonRadius`, biegt sonst die Richtung graduell zum Zentrum

Interactables/Checkpoint.cs

Speichert den Levelfortschritt beim ersten Treffer; ein späterer Neustart respawnt ab hier statt das Level neu zu laden.

Wichtige Code-Elemente:

- Klasse: `Checkpoint` : `LightInteractable`
- Felder: `resetAmmoOnRespawn`, `activatedColor`
- `ProcessRay`: ruft `CheckpointManager.SetCheckpoint` auf, färbt Sprite um, gibt `PassThrough()` zurück

Interactables/ColorFilter.cs

Physikalisch motivierter Farbfilter: kanalweise Multiplikation der Strahlfarbe mit der Filterfarbe plus Beer-Lambert-Dämpfung nach Absorptionskoeffizient/Dicke. Zu wenig/falsche Farbe → vollständige Absorption.

Wichtige Code-Elemente:

- Klasse: `ColorFilter` : `LightInteractable`
- Felder: `filterColor`, `absorptionCoefficient`, `thickness`, `minTransmittedIntensity`
- `ProcessRay`: berechnet transmittierte Farbe & Beer-Lambert-Transmittanz; bei zu geringer Intensität `Absorb()`, sonst `Redirect()` mit neuer Farbe/Intensität

Interactables/ColorSelectiveMirror.cs

Spiegelt nur Strahlen, deren Farbe (mit Toleranz) zu `reflectColor` passt; alles andere passiert ungehindert.

Wichtige Code-Elemente:

- Klasse: `ColorSelectiveMirror` : `LightInteractable`
- Felder: `reflectColor`, `tolerance`, `reflectivity`
- `ProcessRay`: prüft Farbübereinstimmung (`ColorMatches`); bei Match Reflexion via `OpticsMath.Reflect`, sonst `PassThrough()`

Interactables/ConvergingLens.cs

Sammellinse mit zwei Modi: idealisierte Fokussierung auf einen Brennpunkt (für präzises Level-Design) oder physikalisch reale Brechung via Snell'schem Gesetz mit `OpticalMaterial`.

Wichtige Code-Elemente:

- Klasse: `ConvergingLens` : `LightInteractable`
- Felder: `useIdealizedFocus`, `focalPoint`, `focalDistance`, `bendStrength`, `material` (`OpticalMaterial`)
- `ProcessRay`: idealisiert per `Vector2.Lerp` zum Brennpunkt, oder real via `OpticsMath.Reflect` (Fallback: Reflexion bei Totalreflexion)

Interactables/Crystal.cs

Multifunktionaler Kristall mit fünf Modi (Enum `CrystalMode`): Intensität verstärken, fest umlenken, zwischenspeichern & verzögert freigeben, aufspalten, Farbe ändern.

Wichtige Code-Elemente:

- Enum: `CrystalMode` (`Amplify`, `Redirect`, `StoreAndRelease`, `MultiRay`, `ColorChange`)
- Klasse: `Crystal` : `LightInteractable`
- Felder: `mode`, `amplifyFactor`, `redirectAngle`, `storeDuration`, `rayPrefab`, `releaseSpeed`, `rayCount`, `spreadAngle`, `newColor`
- `ProcessRay`: Switch über `mode`; `StoreAndRelease` startet Coroutine `ReleaseAfterDelay`, die nach Verzögerung per `LightRay.CreateAndInitialize` einen neuen Strahl erzeugt

Interactables/DestructibleWall.cs

Wand, die eine begrenzte Anzahl Treffer übersteht, verblasst mit Schaden und wird bei 0 Trefferpunkten zerstört.

Wichtige Code-Elemente:

- Klasse: `DestructibleWall` : `LightInteractable`
- Feld: `hitPoints`
- `ProcessRay`: dekrementiert Trefferpunkte, passt Alpha an, zerstört Objekt bei 0; gibt immer `Absorb()` zurück

Interactables/DivergingLens.cs

Physikalisch basierte Zerstreulinse: bricht mit invertierter lokaler Normale (konkave Eintrittsfläche), wodurch Strahlen auseinanderlaufen statt sich zu sammeln.

Wichtige Code-Elemente:

- Klasse: `DivergingLens` : `LightInteractable`
- Feld: `material` (`OpticalMaterial`)
- `ProcessRay`: `OpticsMath.Reflect` mit invertierter Normale, Fallback Reflexion bei Totalreflexion

Interactables/EnergyField.cs

Bereich, der einen Strahl beim Eintritt verändert (blockieren, verlangsamen, beschleunigen, Farbe/Intensität ändern) oder unverändert durchlässt, wenn deaktiviert.

Wichtige Code-Elemente:

- Enum: EnergyFieldMode (Block, SlowDown, SpeedUp, ChangeColor, ChangeIntensity)
- Klasse: EnergyField : LightInteractable
- Felder: mode, isEnabled, speedMultiplier, fieldColor, intensityMultiplier
- Methode: SetEnabled(bool) – externe Aktivierung (z. B. via Switch)
- ProcessRay: Switch über mode, deaktiviert → PassThrough()

Interactables/FragileMirror.cs

Reflektiert wie ein normaler Spiegel, übersteht aber nur begrenzt viele Treffer, bevor er zerspringt.

Wichtige Code-Elemente:

- Klasse: FragileMirror : LightInteractable
- Felder: hitPoints, glossiness
- ProcessRay: Reflexion via OpticsMath.Reflect (+ zufällige Streuung bei Glossiness), dekrementiert Trefferpunkte, zerstört Objekt verzögert bei 0

Interactables/IntensityGate.cs

Wie ein Switch, aktiviert sich aber nur, wenn die Strahlintensität mindestens minIntensity beträgt.

Wichtige Code-Elemente:

- Klasse: IntensityGate : LightInteractable
- Felder: minIntensity, linkedDoors (List<Door>)
- Property: IsActivated
- ProcessRay: öffnet verknüpfte Türen bei ausreichender Intensität, gibt immer PassThrough() zurück

Interactables/LightAbsorber.cs

Zerstört jeden Strahl, der ihn berührt (Levelbegrenzungen, Sackgassen).

Wichtige Code-Elemente:

- Klasse: LightAbsorber : LightInteractable
- ProcessRay: gibt immer Absorb() zurück

Interactables/LightAccelerator.cs

Erhöht dauerhaft die Geschwindigkeit eines durchtretenden Strahls und verlängert dessen maxRange entsprechend.

Wichtige Code-Elemente:

- Klasse: LightAccelerator : LightInteractable
- Felder: speedMultiplier, extraRange
- ProcessRay: ray.SetSpeed(...), erhöht ray.maxRange, gibt PassThrough() zurück

Interactables/Mirror.cs

Der Standard-Spiegel-Mechanismus: reflektiert um die Oberflächennormale, optionale Glossiness-Streuung, Reflectivity + Fresnel-Schlick-Näherung für winkelabhängigen Intensitätsverlust.

Wichtige Code-Elemente:

- Klasse: Mirror : LightInteractable

- Felder: `reflectivity`, `glossiness`
- `ProcessRay`: `OpticsMath.Reflect` + optionale Streuung + `OpticsMath.FresnelSchlick` für winkelabhängige Intensität, gibt `Redirect()` zurück

Interactables/Portal.cs

Wie ein Teleporter, aber visuell eigenständig und kann optional die Austrittsrichtung fest überschreiben statt sie beizubehalten.

Wichtige Code-Elemente:

- Klasse: `Portal` : `LightInteractable`
- Felder: `linkedPortal`, `overrideExitDirection`, `exitDirection`, `exitOffset`
- `ProcessRay`: berechnet Austrittsposition relativ zum Partnerportal, gibt `InteractionResult.Teleport(...)` zurück

Interactables/Prism.cs

Spaltet weißes Licht in rote/grüne/blau Kindstrahlen mittels chromatischer Dispersion (unterschiedlicher Brechungsindex pro Farbe über Snellius). Bereits farbige Strahlen passieren unverändert.

Wichtige Code-Elemente:

- Klasse: `Prism` : `LightInteractable`
- Felder: `material` (`OpticalMaterial` mit `dispersionOffsetRGB`), `colorTolerance`
- `ProcessRay`: erzeugt 3 Kindstrahlen (Rot/Blau gebrochen, Grün als Referenz ungebrochen) via `RefractOrKeep`, gibt `InteractionResult.Split(...)` zurück
- Hilfsmethoden: `RefractOrKeep`, `IsApproximatelyWhite`

Interactables/Sensor.cs

Aktiviert verknüpfte Lamp-Objekte bei jedem Treffer, lässt den Strahl unverändert weiterlaufen (keine Einmal-Sperre).

Wichtige Code-Elemente:

- Klasse: `Sensor` : `LightInteractable`
- Feld: `linkedLamps` (`List<Lamp>`)
- `ProcessRay`: ruft `lamp.Activate()` für alle verknüpften Lampen auf, gibt `PassThrough()` zurück

Interactables/Splitter.cs

Spaltet einen Strahl fächerförmig in mehrere Strahlen gleicher Farbe auf (im Gegensatz zum Prism, das nach Wellenlänge trennt).

Wichtige Code-Elemente:

- Klasse: `Splitter` : `LightInteractable`
- Felder: `rayCount`, `spreadAngle`
- `ProcessRay`: nutzt geerbtes `BuildFan`, gibt `InteractionResult.Split(...)` zurück

Interactables/Switch.cs

Aktiviert sich bei Treffer (Latch- oder Toggle-Modus) und steuert eine Vielzahl verknüpfter Objekte: Türen, versteckte Objekte, Teleporter, Energiefelder, bewegliche Objekte.

Wichtige Code-Elemente:

- Klasse: `Switch` : `LightInteractable`
- Felder: `toggleMode`, `inactiveColor/activeColor`, `linkedDoors`, `revealTargets`, `linkedTeleporters`, `linkedEnergyFields`, `linkedMovers`
- Property: `IsActivated`

- Methoden: `SetActive` (privat, steuert alle verknüpften Objekte + Sound), `ApplyVisual` (privat)
- `ProcessRay`: schaltet/toggelt Aktivierungszustand, gibt `PassThrough()` zurück

Interactables/Teleporter.cs

Gepaarte Teleporter: ein eintretender Strahl tritt sofort beim Partner aus, Richtung/Geschwindigkeit bleiben erhalten. Zur Laufzeit an-/abschaltbar.

Wichtige Code-Elemente:

- Klasse: `Teleporter : LightInteractable`
- Felder: `linkedTeleporter`, `isEnabled`, `exitOffset`
- Methode: `SetEnabled(bool)`
- `ProcessRay`: berechnet Austrittsposition relativ zum Partner, gibt `InteractionResult.Teleport(...)` zurück, oder `PassThrough()` wenn deaktiviert

Ordner: Gameplay

Gameplay/LevelGoal.cs

Zielpunkt des Levels: Erreicht ein `LightRay` das Ziel, wird der Strahl beendet, der Sieg registriert und eine Pulsier-Animation abgespielt, gefolgt vom Level-Complete-Bildschirm.

Wichtige Code-Elemente:

- Klasse: `LevelGoal` (`MonoBehaviour`)
- Felder: `winColor`, `pulseDuration`, `pulseCount`
- Methoden: `OnTriggerEnter2D/OnCollisionEnter2D`; `HandleWin` (terminiert Strahl, benachrichtigt `ShotBudget`, `Sound/VFX`); `WinAndGlow` (`Coroutine`, `Farbanimation` + zeigt `LevelCompleteController`)

Gameplay/LightSource.cs

Die vom Spieler gesteuerte Lichtquelle: Linksklick feuert einen `LightRay` in Richtung der Mausposition, sofern Munition im `ShotBudget` verfügbar ist. Zentrale Eingabekomponente des Spiels.

Wichtige Code-Elemente:

- Klasse: `LightSource` (`MonoBehaviour`)
- Felder: `speed`, `prefabToSpawn`, `cam`, `shotBudget`
- Methoden: `Update` (erkennt Klick); `Fire` (privat, erzeugt Strahl via `LightRay.CreateAndInitialize`); `GetMouseWorldPosition` (privat)

Gameplay/ShotBudget.cs

Begrenzt die Anzahl abfeuerbarer Lichtstrahlen pro Level, benachrichtigt die HUD per Event und prüft nach dem letzten Schuss, ob der Fail-Screen gezeigt werden muss.

Wichtige Code-Elemente:

- Klasse: `ShotBudget` (`MonoBehaviour`)
- Felder/Properties: `maxShots`, `Remaining`, `Used`, `OnShotsChanged` (event)
- Methoden: `TryConsumeShot`; `NotifyLevelWon`; `RestoreRemaining`; `CheckForFailAfterLastShot` (`Coroutine`)

Ordner: Level

Level/CheckpointManager.cs

Merkt sich den zuletzt erreichten Checkpoint; ermöglicht Respawn statt vollständigem Szenen-Reload.

Wichtige Code-Elemente:

- Klasse: CheckpointManager (MonoBehaviour)
- Property: HasCheckpoint
- Methoden: SetCheckpoint; RespawnAtCheckpoint

Level/Door.cs

Blockiert Licht solange geschlossen, lässt es passieren wenn offen; wird typischerweise von einem Switch gesteuert.

Wichtige Code-Elemente:

- Klasse: Door : LightInteractable
- Felder: startsOpen, closedColor, openColor
- Property: IsOpen
- Methoden: SetOpen(bool)
- ProcessRay: PassThrough() wenn offen, sonst Absorb()

Level/FogOfWarManager.cs

Statische, zentrale Registrierung aller FogTile-Objekte; "Aufdecker" rufen RevealCircle auf, um Tiles in einem Radius dauerhaft aufzudecken.

Wichtige Code-Elemente:

- Klasse: FogOfWarManager (static)
- Methoden: Register, Unregister, RevealCircle(worldPosition, radius, fadeDuration)

Level/FogTile.cs

Eine einzelne Zelle des Fog-of-War-Rasters; startet undurchsichtig und verblasst einmalig beim Aufdecken.

Wichtige Code-Elemente:

- Klasse: FogTile (MonoBehaviour)
- Property: IsRevealed
- Methoden: Reveal(fadeDuration); FadeOut (Coroutine, privat)

Level/HiddenObject.cs

Macht ein GameObject unsichtbar/nicht-interagierbar, bis Reveal() aufgerufen wird (z. B. durch Switch/Sensor).

Wichtige Code-Elemente:

- Klasse: HiddenObject (MonoBehaviour)
- Methoden: Reveal; SetVisible (privat, de/aktiviert SpriteRenderer & Collider2D)

Level/Lamp.cs

Lampe, die aktiviert wird (typischerweise durch Sensor): deckt Fog-of-War um sich herum auf und schaltet Sprite-Farbe sowie optionales Light2D frei.

Wichtige Code-Elemente:

- Klasse: Lamp (MonoBehaviour)

- Felder: revealRadius, fadeDuration, litColor
- Methode: Activate (einmalig via Guard-Flag)

Level/MovingObject.cs

Bewegt sein GameObject im Ping-Pong-Verfahren zwischen zwei relativen Offsets zur Startposition, per kinematischem Rigidbody2D.

Wichtige Code-Elemente:

- Klasse: MovingObject (MonoBehaviour, DisallowMultipleComponent)
- Felder: pointA, pointB, speed, isMoving
- Methoden: SetMoving(bool); FixedUpdate (Mathf.PingPong-Bewegung)

Level/PlayerRotatable.cs

Ermöglicht dem Spieler, ein Objekt (z. B. Spiegel) per Mausexplorer zu drehen, solange der Mauszeiger darüber ist.

Wichtige Code-Elemente:

- Klasse: PlayerRotatable (MonoBehaviour)
- Feld: degreesPerScrollNotch
- Methoden: OnMouseEnter/OnMouseExit; Update (Mausexplorer-Rotation)

Level/Pulsate.cs

Sinusförmige Pulsation der Intensität eines Light2D für einen lebendigeren "flackernden Licht"-Effekt auf statischen beleuchteten Objekten.

Wichtige Code-Elemente:

- Klasse: Pulsate (MonoBehaviour)
- Felder: speed, amount (0–1)
- Verhalten: speichert Basisintensität in Awake, moduliert sie fortlaufend in Update

Level/RotatingObject.cs

Rotiert sein GameObject kontinuierlich mit konstanter Winkelgeschwindigkeit (typischerweise ein Spiegel).

Wichtige Code-Elemente:

- Klasse: RotatingObject (MonoBehaviour)
- Felder: degreesPerSecond, isRotating
- Methoden: SetRotating(bool); Update

Ordner: UI

UI/GameHUD.cs

Steuert die Kopfzeilen-HUD im Spiel: verbleibende/verwendete Lichtbälle, Levelname, verstrichene Zeit, Pause-Button. Abonniert `ShotBudget.OnShotsChanged`.

Wichtige Code-Elemente:

- Klasse: `GameHUD` (`MonoBehaviour`)
- Felder: `shotBudget`, `pauseMenu`, `remainingText`, `usedText`, `levelNameText`, `timerText`
- Property: `ElapsedSeconds`
- Methoden: `FormatElapsed`; `HandleShotsChanged` (privat); `OnPauseButtonPressed`

UI/LevelCompleteController.cs

Wird von `LevelGoal` angezeigt: erfasst Fortschritt (Sterne, Zeit, Munition) über `ProgressManager` und bietet Wechsel zum nächsten Level oder Hauptmenü.

Wichtige Code-Elemente:

- Klasse: `LevelCompleteController` (`MonoBehaviour`)
- Felder: `panel`, `statsText`
- Methoden: `Show` (pausiert Spiel, berechnet Sterne, speichert Fortschritt); `OnNextLevelButtonPressed`; `OnMainMenuButtonPressed`

UI/LevelFailedController.cs

Wird von `ShotBudget` angezeigt, wenn die Munition aufgebraucht ist, ohne das Ziel zu erreichen; bietet Neustart (ggf. ab Checkpoint) oder Hauptmenü.

Wichtige Code-Elemente:

- Klasse: `LevelFailedController` (`MonoBehaviour`)
- Felder: `panel`, `usedBallsText`, `timeText`
- Methoden: `Show`; `OnRestartButtonPressed` (Checkpoint-Respawn oder Szenen-Reload); `OnMainMenuButtonPressed`

UI/LevelSelectController.cs

Baut zur Laufzeit eine Zeile pro Level aus `LevelIds.AllLevels` auf: Sperr-/Abschlussstatus, Sterne, Bestzeit, beste Munitionsnutzung.

Wichtige Code-Elemente:

- Klasse: `LevelSelectController` (`MonoBehaviour`)
- Felder: `content`, `uiFont`
- Methoden: `BuildRows` (privat); `BuildRow` (privat, pro Level); `CreateLabel` (privat); `OnBackButtonPressed`

UI/MainMenuController.cs

Steuert das Hauptmenü: Menümusik, Buttons für Spiel starten, Levelauswahl, Einstellungen, Beenden.

Wichtige Code-Elemente:

- Klasse: `MainMenuController` (`MonoBehaviour`)
- Methoden: `OnStartButtonPressed`; `OnLevelSelectButtonPressed`; `OnSettingsButtonPressed`; `OnQuitButtonPressed`

UI/PauseMenuController.cs

Öffnet/schließt das Pausemenü per konfigurierbarer Taste (KeyBindings) und pausiert dabei die Spielzeit.

Wichtige Code-Elemente:

- Klasse: PauseMenuController (MonoBehaviour)
- Feld: pausePanel
- Methoden: TogglePause; OnResumeButtonPressed; OnRestartButtonPressed; OnMainMenuButtonPressed

UI/SceneLoader.cs

Statische Hilfsklasse für Szenenwechsel; setzt `Time.timeScale` bei jedem Wechsel zurück.

Wichtige Code-Elemente:

- Klasse: SceneLoader (static)
- Methoden: LoadScene; ReloadCurrentScene; QuitApplication

UI/SettingsController.cs

Steuert den Einstellungsbildschirm: Musik-/SFX-Lautstärke, Vollbild, Auflösung, Helligkeit, Sprache, Tasten-Neubelegung, Fortschritt zurücksetzen.

Wichtige Code-Elemente:

- Klasse: SettingsController (MonoBehaviour)
- Felder: musicSlider, sfxSlider, brightnessSlider, fullscreenToggle, resolutionDropdown, languageDropdown, pauseKeyLabel
- Methoden: SetupAudioControls/Brightness/Fullscreen/Resolutions/Language (privat, Setup); SetBrightness; SetFullscreen; SetResolution; SetLanguage; OnRebindPauseKeyButtonPressed; OnResetProgressButtonPressed; OnBackButtonPressed